

طراحی الگوریتم‌ها – مرتب‌سازی با هیپ

Introduction to Algorithm

مهدی جوانمردی

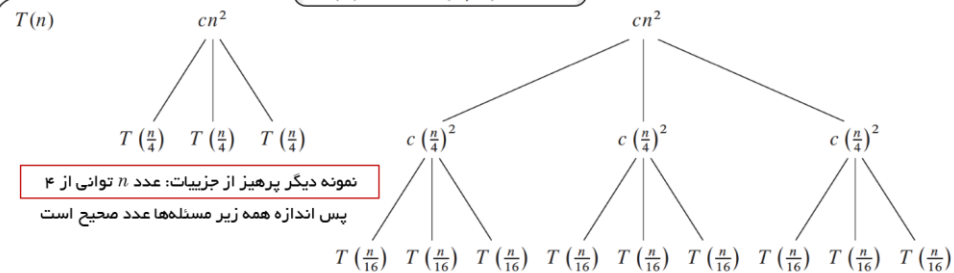
مرور جلسه قبل

حل رابطه بازگشتی با درخت بازگشتی

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$$

نمونه پرهیز از جزییات: محاسبه حد بالا با حذف کف

$$T(n) = 3T(n/4) + cn^2$$



اثبات حدس به دست آمده از طریق استقرائ

حل روابط بازگشت با قضیه اصلی

$$T(n) = aT(n/b) + f(n) \quad a \geq 1 \text{ and } b > 1$$

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. شرط منظم بودن

$$f(n) \square n^{\log_b a}$$

چالش‌های روش جایگذاری

حدس $T(n) = O(n) \rightarrow T(n) \leq cn \quad \times \rightarrow T(n) \leq cn - d, \text{ where } d \geq 0 \text{ is a constant.}$

۱. درجه جمله اضافه کمتر از حکم باشد: به حکم یک جمله از درجه کمتر اضافه میکنیم
۲. درجه جمله اضافه با حکم برابر باشد: یک فاکتور لگاریتم در حکم کمتر حدس زدیم
۳. درجه جمله اضافه بیشتر از حکم باشد: باید حکم از درجه بالاتری باشد

تغییر متغیر برای حل رابطه بازگشتی

$$\left. \begin{array}{l} T(n) = 2T(\sqrt{n}) + \lg n \\ m = \lg n \end{array} \right\} \left. \begin{array}{l} T(2^m) = 2T(2^{m/2}) + m \\ S(m) = T(2^m) \end{array} \right\} \begin{array}{l} S(m) = 2S(m/2) + m \\ \downarrow \\ S(m) = O(m \lg m) \\ \downarrow \\ T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n) \end{array}$$

فصل ششم: مرتب‌سازی هرمی | Heap sort

• مرتب‌سازی هرمی یا Heap sort

• Heap چیست

• نگهداری خاصیت Heap

• درست کردن Heap

• الگوریتم مرتب‌سازی هرمی

• صف‌های اولویت

II Sorting and Order Statistics

Introduction 147

6 Heapsort 151

6.1 Heaps 151

6.2 Maintaining the heap property 154

6.3 Building a heap 156

6.4 The heapsort algorithm 159

6.5 Priority queues 162

7 Quicksort 170

7.1 Description of quicksort 170

7.2 Performance of quicksort 174

7.3 A randomized version of quicksort 179

7.4 Analysis of quicksort 180



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

مرتب‌سازی و ساختار داده‌ای

- در واقعیت اعدادی که نیاز به مرتب‌سازی دارند معمولاً اعداد منفرد نیستند

2983

0372

8746

2946

Key

Record: Satellite image

- در عمل در هنگام جابجایی Key توسط الگوریتم مرتب‌سازی record نیز باید جابجا شود

- اگر حجم داده record زیاد باشد آرایه‌ای از اشاره‌گرها به داده جابجا میشوند و نه خود داده

مقایسه الگوریتم‌های مرتب‌سازی

only a constant number of elements of the input are ever stored outside the array.

fast in-place sorting algorithm for small input

sorts *in place*

• الگوریتم Insertion sort

worst-case running time $\Theta(n^2)$

مرتبه زمانی

expected running time $\Theta(n^2)$

• الگوریتم Merge sort

running time $\Theta(n \lg n)$

مرتبه زمانی

important data structure, called a heap priority queue

sorts *in place*

• الگوریتم Heap sort

worst-case running time $O(n \lg n)$

مرتبه زمانی

quicksort has tight code.
popular algorithm for sorting large input arrays

sorts *in place*

• الگوریتم Quick sort

worst-case running time $\Theta(n^2)$

مرتبه زمانی

expected running time $\Theta(n \lg n)$

outperforms heapsort in practice

مقایسه الگوریتم‌های مرتب‌سازی

we can beat this lower bound of $\Omega(n \lg n)$

if we can gather information about the sorted order of the input

• الگوریتم Counting sort

the input numbers are in the set $\{0, 1, \dots, k\}$

worst-case running time $\Theta(k + n)$

expected running time $\Theta(k + n)$

مرتبه زمانی

• الگوریتم Radix sort

there are n integers to sort

integer has d digits

digit can take on up to k possible values

worst-case running time $\Theta(d(n + k))$

expected running time $\Theta(d(n + k))$

مرتبه زمانی

• الگوریتم Bucket sort

requires knowledge of the probabilistic
distribution of numbers in the input array

real numbers uniformly distributed in the half-open interval $[0, 1)$

worst-case running time $\Theta(n^2)$

average-case running time $\Theta(n)$

مرتبه زمانی

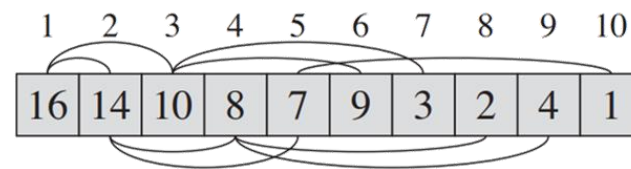
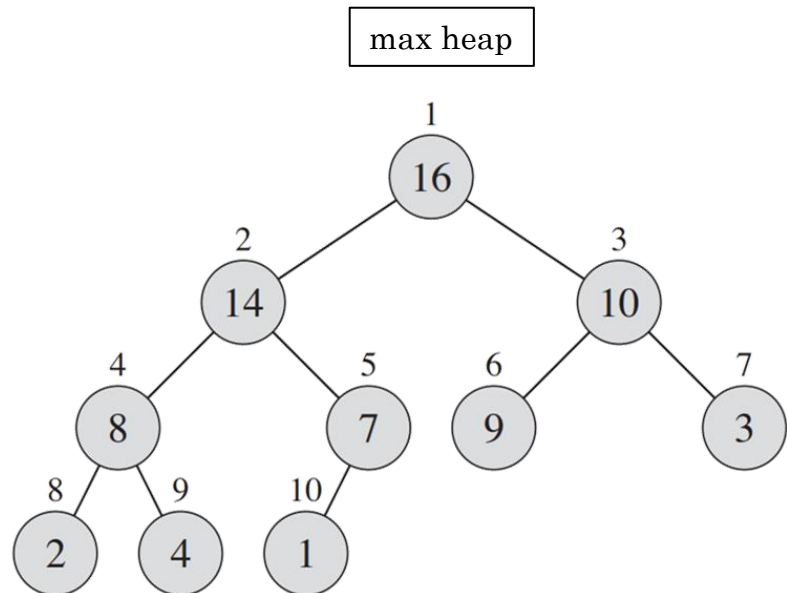
ساختار Heap و ویژگی‌های آن - ۱

• Heap عبارت است از یک درخت دودویی کامل (به غیر از پایین‌ترین سطح)

• هر گره از درخت یک المان از آرایه

• max heap: هر گره بزرگتر مساوی فرزندان $A[\text{PARENT}(i)] \geq A[i]$

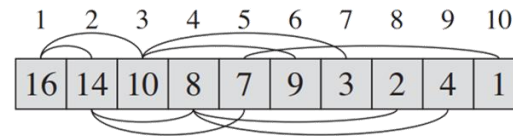
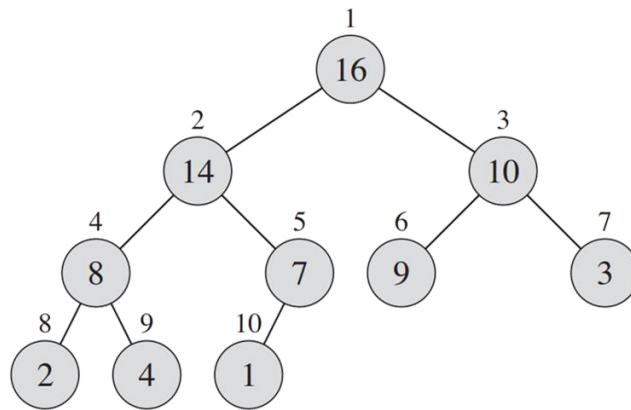
• min heap: هر گره کوچکتر مساوی فرزندان $A[\text{PARENT}(i)] \leq A[i]$



root: $A[1]$
 node: $A[i]$
 left-child: $A[2i]$
 right-child: $A[2i+1]$
 leaves: $A[(\lfloor n/2 \rfloor + 1) .. n]$

ساختار Heap و ویژگی‌های آن - ۲

• Heap عبارت است از یک درخت دودویی کامل



```
PARENT(i)
1  return  $\lfloor i/2 \rfloor$ 

LEFT(i)
1  return  $2i$ 

RIGHT(i)
1  return  $2i + 1$ 
```

HEAP: array A {

- $A.length$: تعداد اعداد در آرایه $A[1 \dots A.length]$
- $A.heap_size$: چه تعداد المان‌های هرم در آرایه مرتب شده است
 $A[1 \dots A.heap_size]$, where $0 \leq A.heap_size \leq A$

عملیات‌های Heap

Maintain/Restore the max-heap property

MAX-HEAPIFY

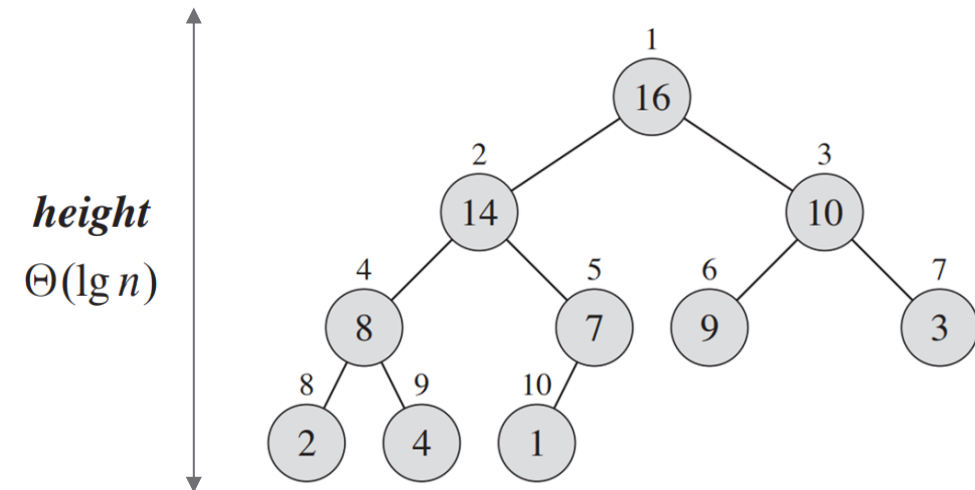
Create a max-heap from an unordered array

BUILD-MAX-HEAP

Sort an array in place

HEAPSORT

Priority queues

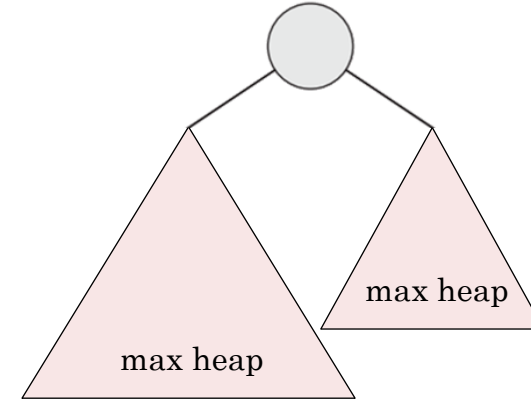
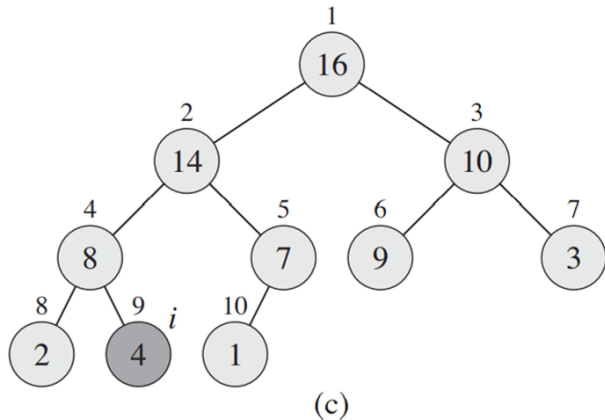
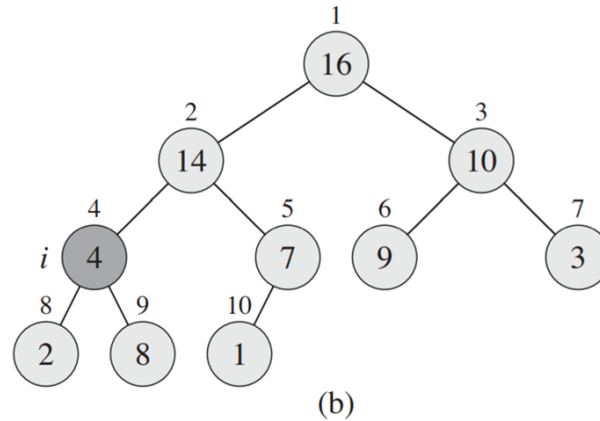
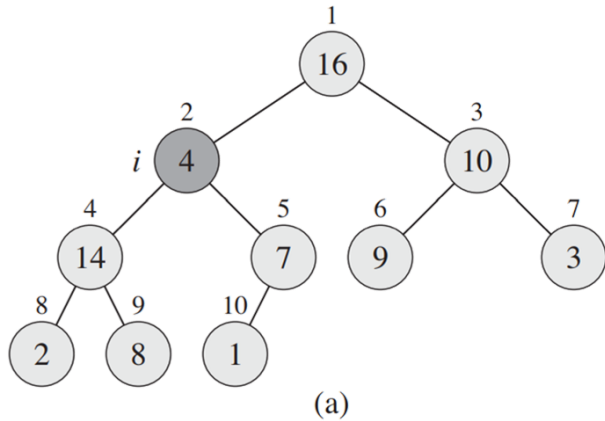


نگهداری ویژگی heap با عملیات MAX-HEAPIFY

مثال

MAX-HEAPIFY($A, 2$)

$A.heap-size = 10$



MAX-HEAPIFY(A, i)

- 1 $l = \text{LEFT}(i)$
- 2 $r = \text{RIGHT}(i)$
- 3 **if** $l \leq A.heap-size$ and $A[l] > A[i]$
- 4 $largest = l$
- 5 **else** $largest = i$
- 6 **if** $r \leq A.heap-size$ and $A[r] > A[largest]$
- 7 $largest = r$
- 8 **if** $largest \neq i$
- 9 exchange $A[i]$ with $A[largest]$
- 10 MAX-HEAPIFY($A, largest$)

تحليل زمني عمليات MAX-HEAPIFY

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

$\Theta(1)$

$T(2n/3)$

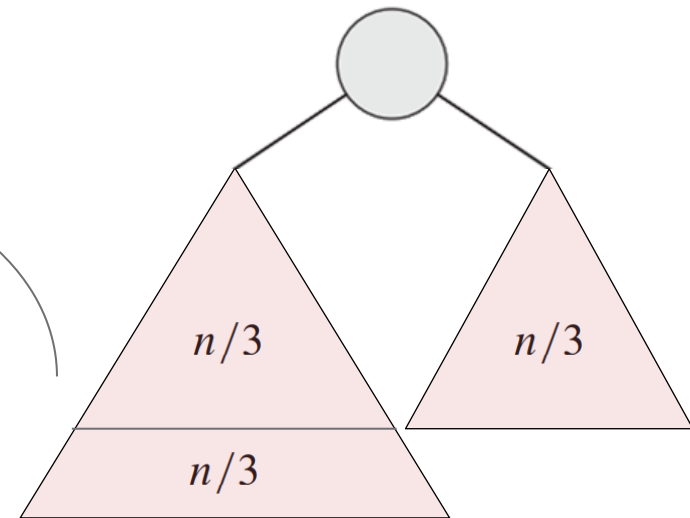
$$T(n) \leq T(2n/3) + \Theta(1)$$

case 2 of
the master theorem

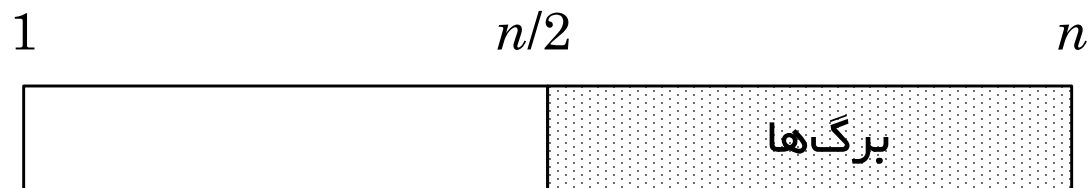
$$T(n) = O(\lg n)$$

running time of MAX-HEAPIFY on a node of height h as $O(h)$

worst case

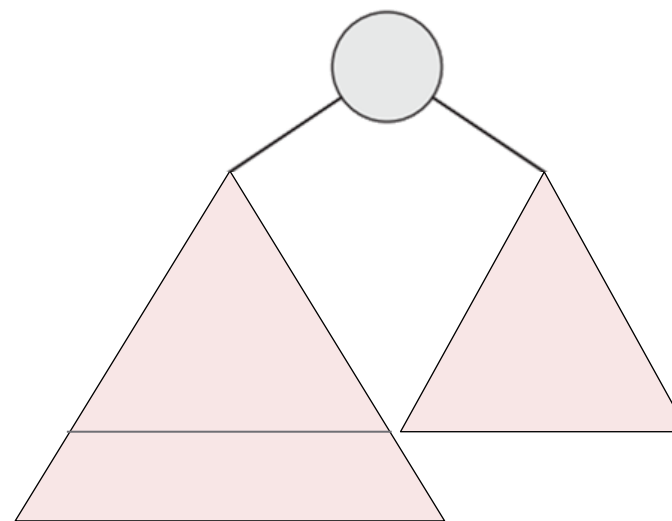


درست کردن heap



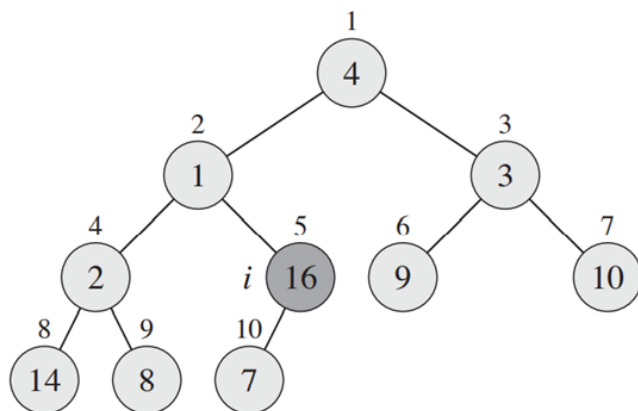
BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

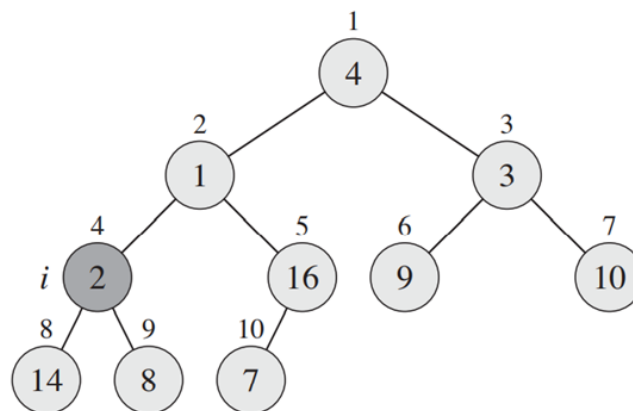


درست کردن heap – مثال

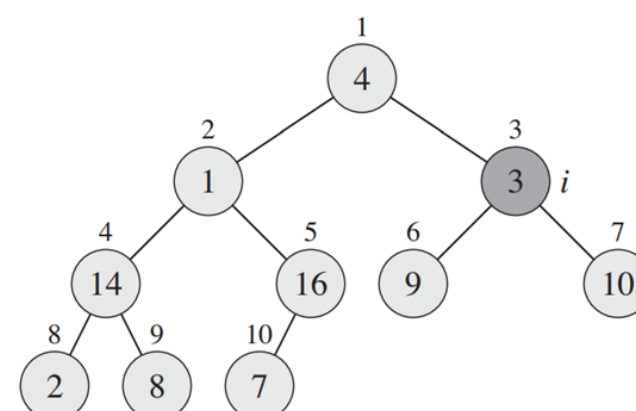
A [4 1 3 2 16 9 10 14 8 7]



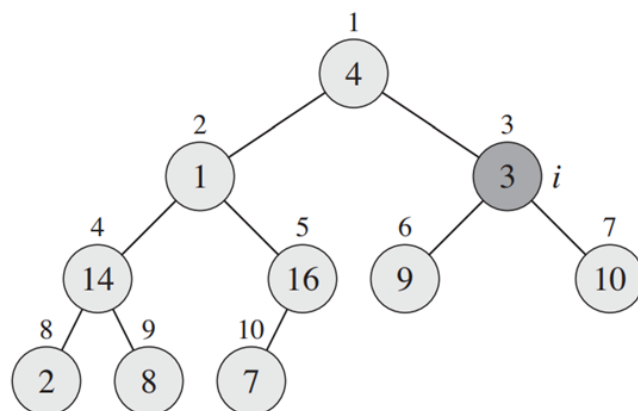
(a)



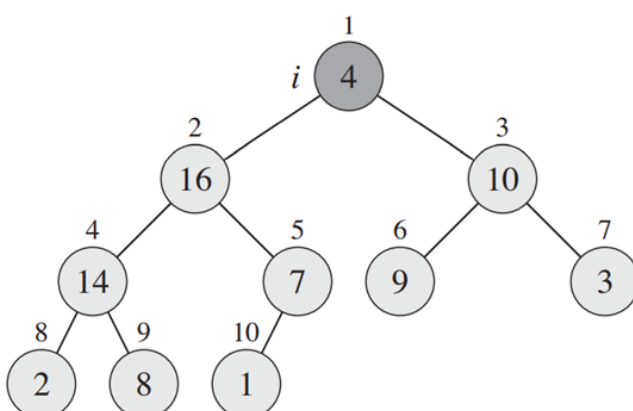
(b)



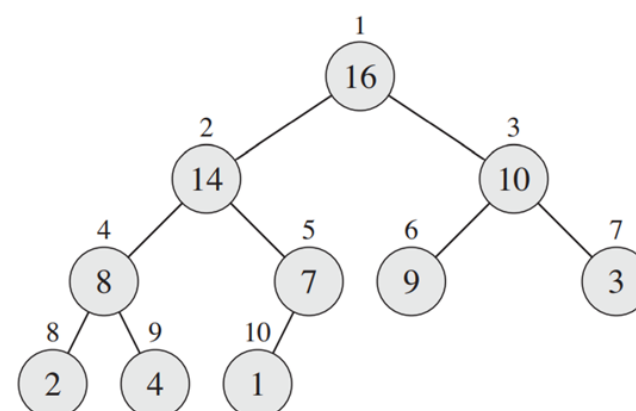
(c)



(c)



(e)



(f)

اثبات الگوریتم BUILD-MAX-HEAP

مستقل از حلقه

BUILD-MAX-HEAP(A)

```
1  $A.heap-size = A.length$   
2 for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3   MAX-HEAPIFY( $A, i$ )
```

At the start of each iteration of the **for** loop of lines 2–3, each node $i + 1, i + 2, \dots, n$ is the root of a max-heap.

Initialization: Prior to the first iteration of the loop, $i = \lfloor n/2 \rfloor$. Each node $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ is a leaf and is thus the root of a trivial max-heap.

Maintenance: To see that each iteration maintains the loop invariant, observe that the children of node i are numbered higher than i . By the loop invariant, therefore, they are both roots of max-heaps. This is precisely the condition required for the call MAX-HEAPIFY(A, i) to make node i a max-heap root. Moreover, the MAX-HEAPIFY call preserves the property that nodes $i + 1, i + 2, \dots, n$ are all roots of max-heaps. Decrementing i in the **for** loop update reestablishes the loop invariant for the next iteration.

Termination: At termination, $i = 0$. By the loop invariant, each node $1, 2, \dots, n$ is the root of a max-heap. In particular, node 1 is.

BUILD-MAX-HEAP تحلیل زمانی

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$ 
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

MAX-HEAPIFY costs $O(\lg n)$ $\times$ BUILD-MAX-HEAP makes $O(n)$ such calls

the running time is $O(n \lg n)$

$$\begin{aligned} T(n) &= \sum_{h=0}^{\lg(n)} \left\lceil \frac{n}{2^{h+1}} \right\rceil * O(h) \\ &= O\left(n * \sum_{h=0}^{\lg(n)} \frac{h}{2^h}\right) \\ &= O\left(n * \sum_{h=0}^{\infty} \frac{h}{2^h}\right) \end{aligned}$$

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$$

for $|x| < 1$.

$$\begin{aligned} &= O\left(n * \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2}\right) \\ &= O(n * 2) \\ &= O(n) \end{aligned}$$

$$\sum_{n=0}^{\infty} x^n = \frac{1}{1-x} \quad \text{for } |x| < 1.$$

$$\sum_{n=1}^{\infty} nx^{n-1} = \frac{d}{dx} \left[\sum_{n=0}^{\infty} x^n \right] = \frac{d}{dx} \left[\frac{1}{1-x} \right] = \frac{1}{(1-x)^2}$$

$$\sum_{n=0}^{\infty} nx^n = \frac{x}{(1-x)^2}$$

الگوریتم heap sort

HEAPSORT(A)

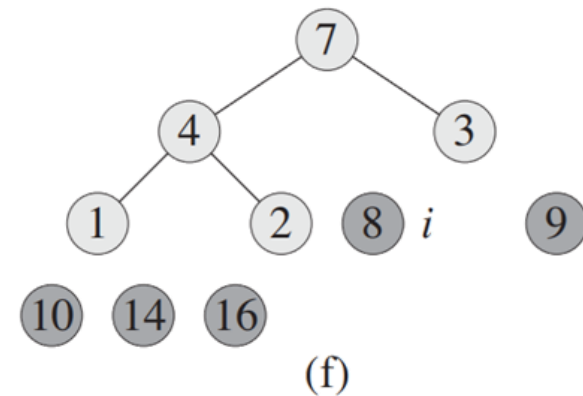
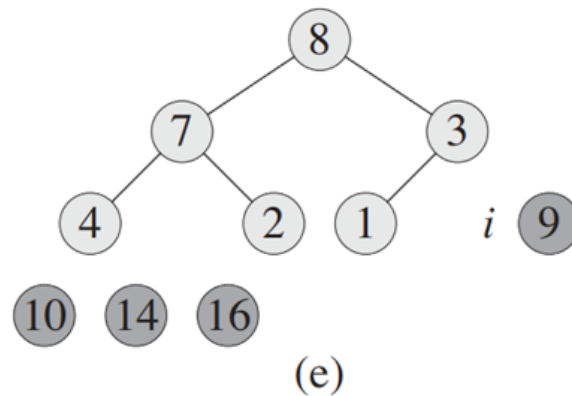
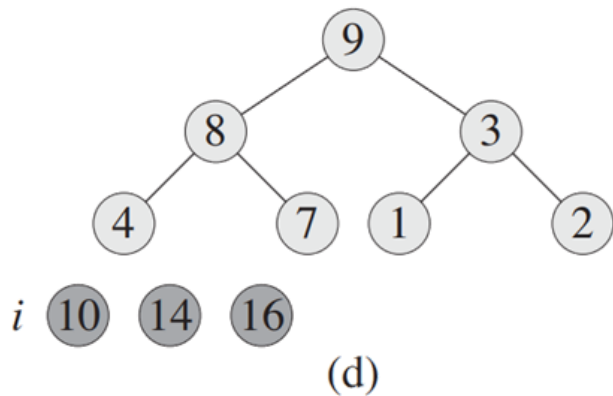
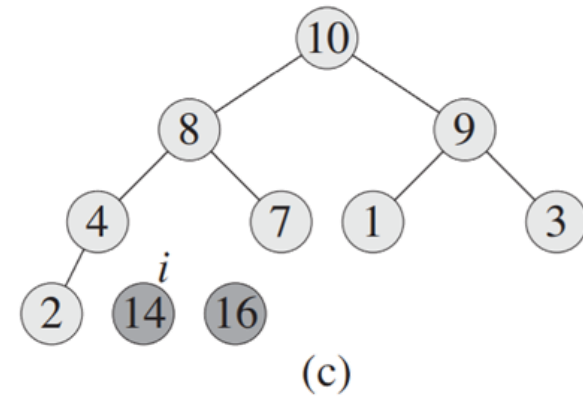
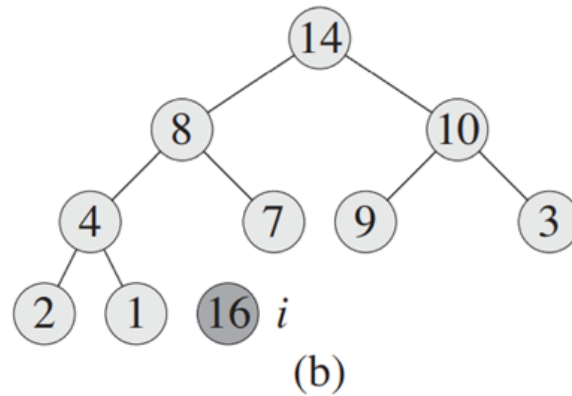
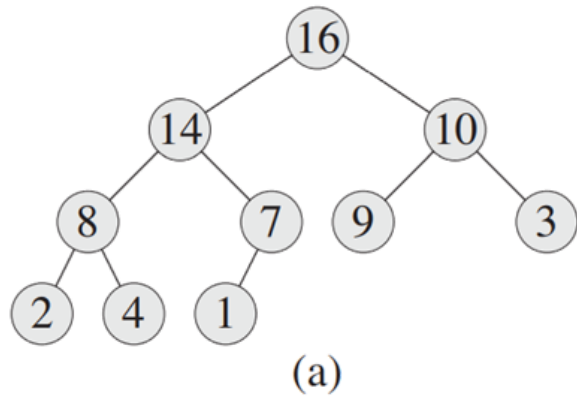
1	BUILD-MAX-HEAP(A)	$O(n)$	} $n \text{ times} = O(n \lg n)$
2	for $i = A.length$ downto 2		
3	exchange $A[1]$ with $A[i]$		
4	$A.heap\text{-}size = A.heap\text{-}size - 1$		
5	MAX-HEAPIFY($A, 1$)	$O(\lg n)$	

HEAPSORT(A)

```

1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap\text{-}size = A.heap\text{-}size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
    
```

مثال heap sort

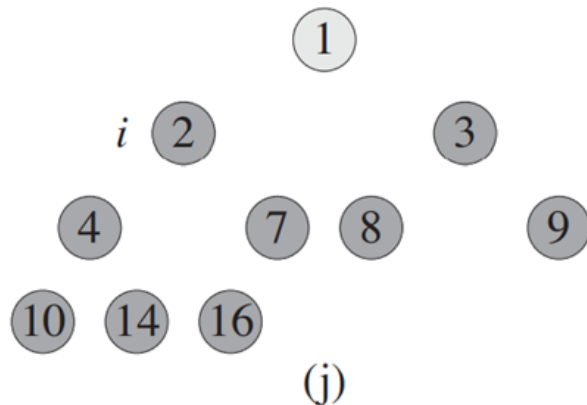
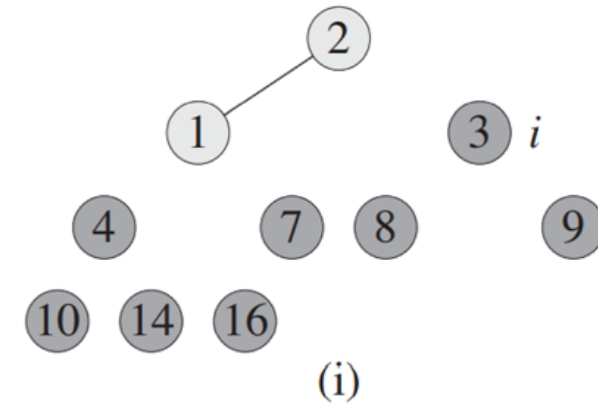
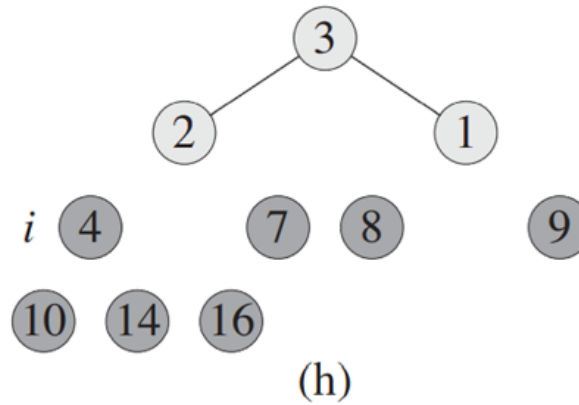
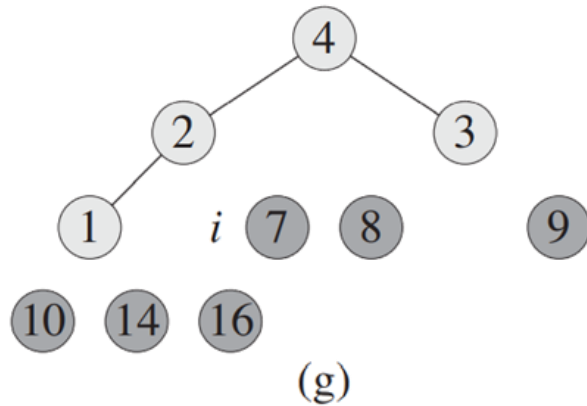


HEAPSORT(A)

```

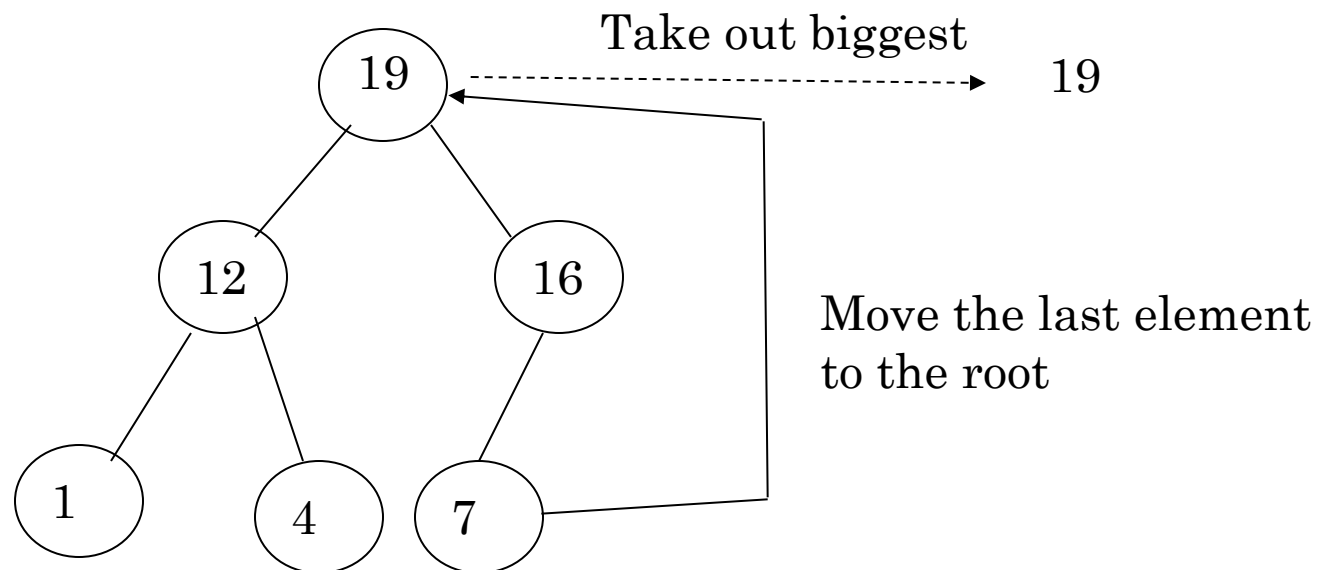
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
    
```

مثال heap sort

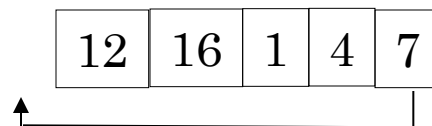


(k)

Example of Heap Sort

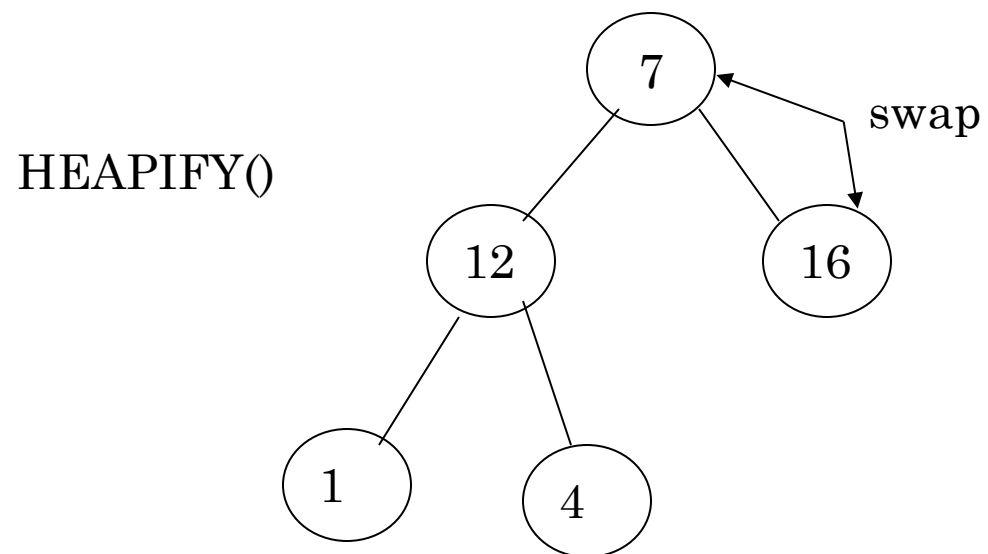


Array A



Sorted:



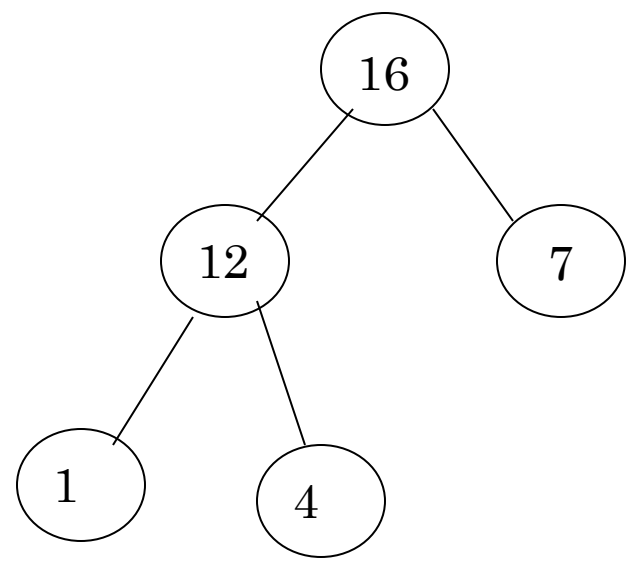


Array A

7	12	16	1	4
---	----	----	---	---

Sorted:

19

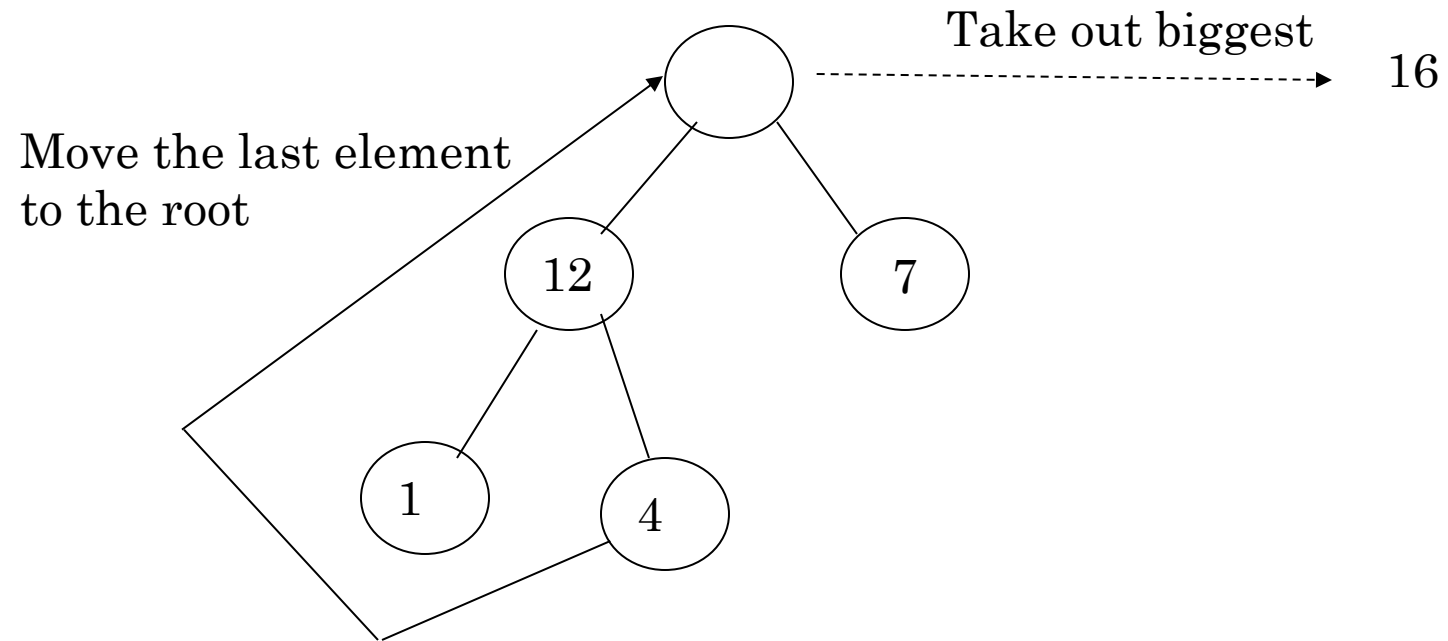


Array A

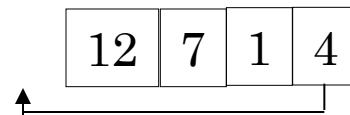
16	12	7	1	4
----	----	---	---	---

Sorted:

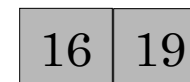
19

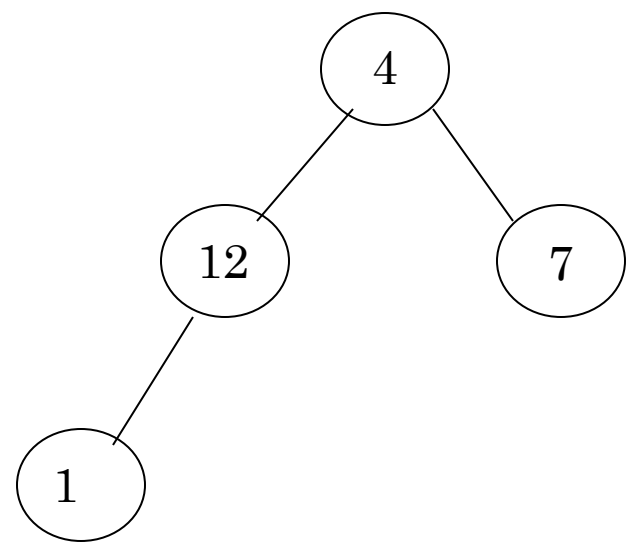


Array A



Sorted:



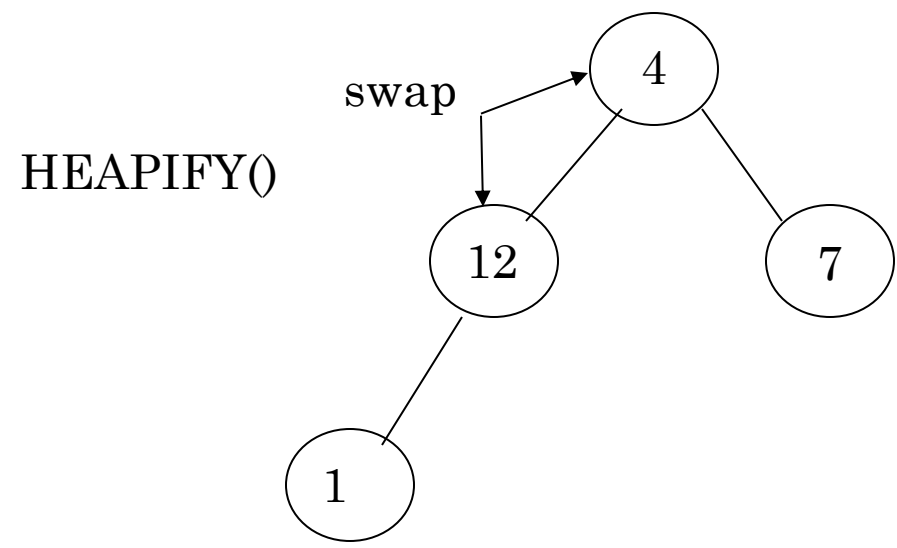


Array A

4	12	7	1
---	----	---	---

Sorted:

16	19
----	----

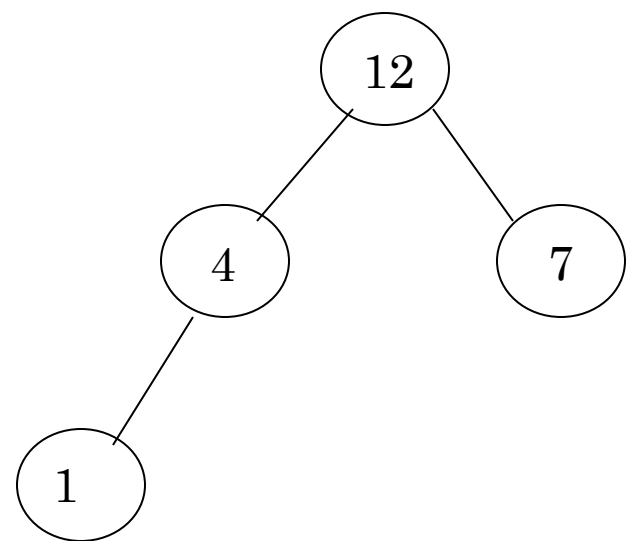


Array A

4	12	7	1
---	----	---	---

Sorted:

16	19
----	----

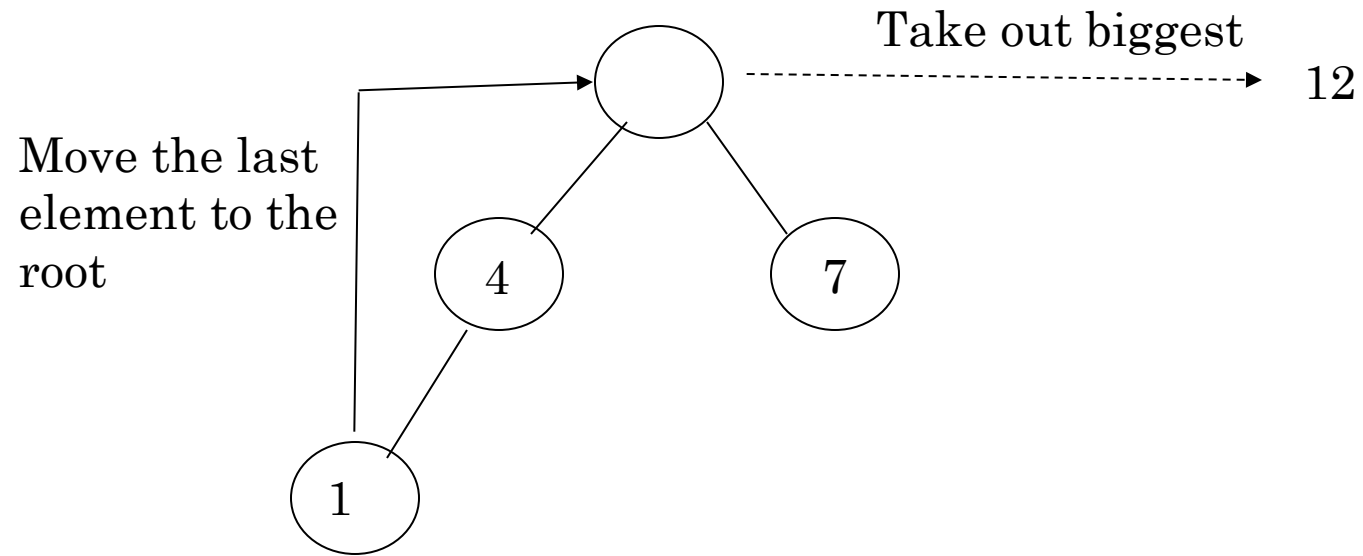


Array A

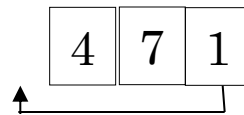
12	4	7	1
----	---	---	---

Sorted:

16	19
----	----

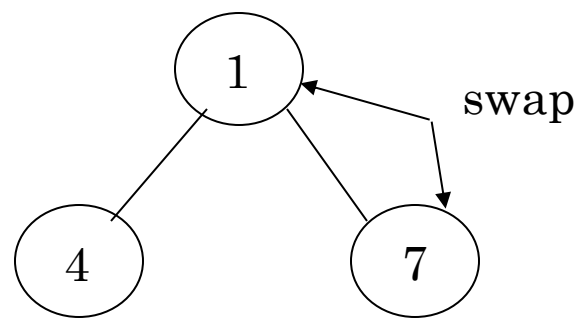


Array A



Sorted:



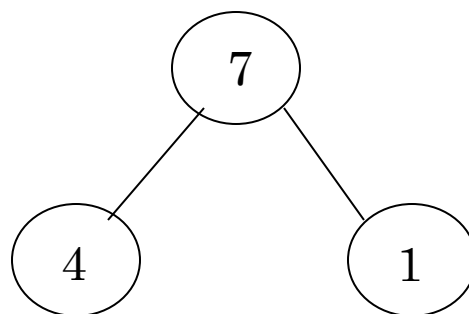


Array A

1	4	7
---	---	---

Sorted:

12	16	19
----	----	----

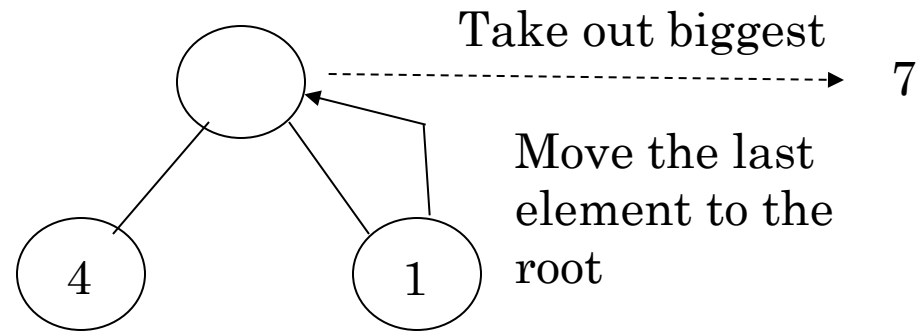


Array A

7	4	1
---	---	---

Sorted:

12	16	19
----	----	----



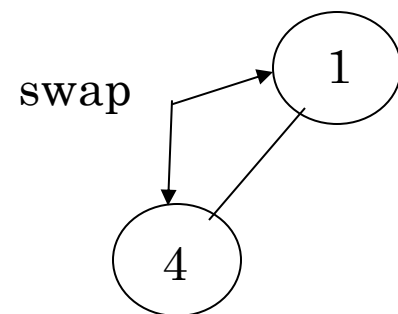
Array A

1	4
---	---

Sorted:

7	12	16	19
---	----	----	----

HEAPIFY()

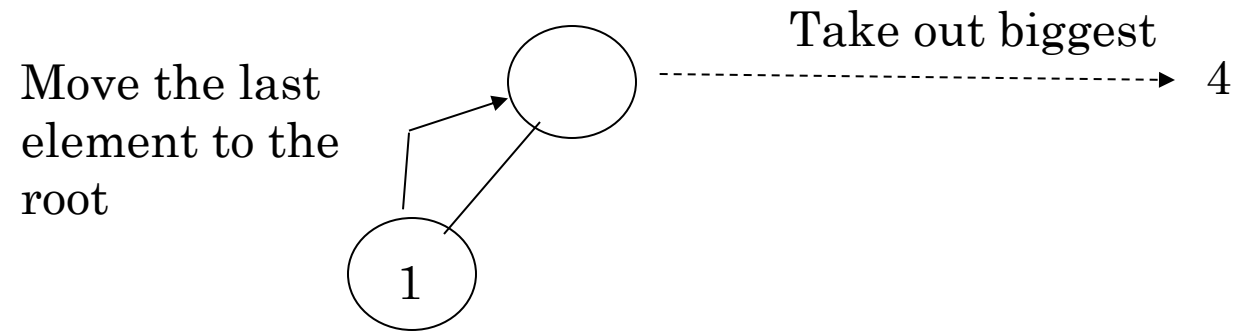


Array A

4	1
---	---

Sorted:

7	12	16	19
---	----	----	----

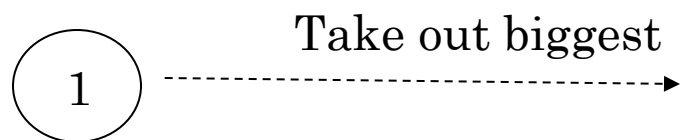


Array A

1

Sorted:

4 7 12 16 19



Array A

Sorted:

1	4	7	12	16	19
---	---	---	----	----	----

Sorted:

1	4	7	12	16	19
---	---	---	----	----	----

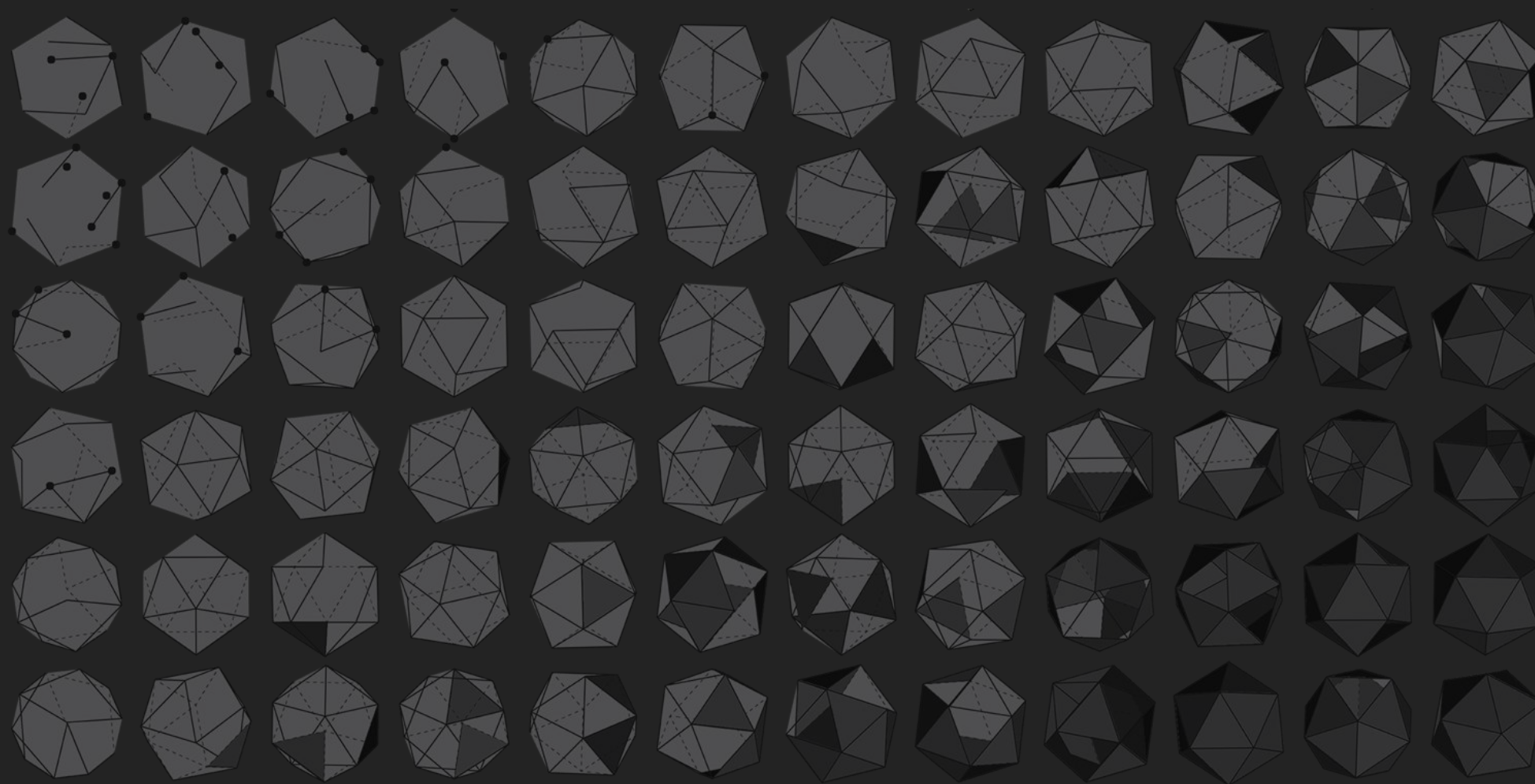
اثبات الگوریتم heap sort

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

مستقل از حلقه

امتیازی! ۳ دقیقه فرصت



صف اولویت و عملیات‌های آن

صف اولویت یا priority queue

A *priority queue* is a data structure for maintaining a set S of elements, each with an associated value called a *key*. A *max-priority queue* supports the following operations:

INSERT(S, x) inserts the element x into the set S , which is equivalent to the operation $S = S \cup \{x\}$.

MAXIMUM(S) returns the element of S with the largest key.

EXTRACT-MAX(S) removes and returns the element of S with the largest key.

INCREASE-KEY(S, x, k) increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key value.

عملیات‌های روی priority queue

HEAP-MAXIMUM(A)

```
1 return  $A[1]$ 
```

HEAP-EXTRACT-MAX(A)

```
1 if  $A.heap-size < 1$   
2   error "heap underflow"  
3  $max = A[1]$   
4  $A[1] = A[A.heap-size]$   
5  $A.heap-size = A.heap-size - 1$   
6 MAX-HEAPIFY( $A, 1$ )  
7 return  $max$ 
```

HEAP-INCREASE-KEY(A, i, key)

```
1 if  $key < A[i]$   
2   error "new key is smaller than current key"  
3  $A[i] = key$   
4 while  $i > 1$  and  $A[PARENT(i)] < A[i]$   
5   exchange  $A[i]$  with  $A[PARENT(i)]$   
6    $i = PARENT(i)$ 
```

MAX-HEAP-INSERT(A, key)

```
1  $A.heap-size = A.heap-size + 1$   
2  $A[A.heap-size] = -\infty$   
3 HEAP-INCREASE-KEY( $A, A.heap-size, key$ )
```